A Log-Normal Model of Server CPU Utilization Under Transaction Load: Fit, Scaling, and SLA-Aware Sizing

Alexander Apartsin
apartsin@gmail.com

Draft, August 2026

Abstract

Sizing servers for a transaction-processing service is a costly trade-off: over-provisioning wastes infrastructure budget, under-provisioning drops transactions and triggers SLA penalties. Sound sizing rests on one quantity: given a transaction rate, the probability that CPU utilization stays below the level at which service-level objectives are met. We present a probabilistic answer built on a log-normal model of CPU at fixed load combined with Poisson event arrivals, which produces a capacity ceiling and, under a tiered SLA schedule, an expected-revenue estimate. We evaluate the log-normal claim on three public production traces from two providers (Bitbrains GWA-T-12 fastStorage, 1 250 VMs; Bitbrains Rnd, 500 VMs; Alibaba cluster-trace-v2018, 298 sampled machines) totalling 19 624 hour-of-day distribution bins: log-normal is the best-fitting family among {normal, gamma, Weibull, log-normal} by AIC in 66.71% of VM-hour bins (95% cluster-bootstrap CI [64.61, 68.76]). We derive the parameter-scaling law implied by the multiplicative-overhead mechanism, $\sigma_{\log C}(\lambda) = \sqrt{\alpha^2\lambda + \sigma_\varepsilon^2}$, fit it per VM, and extend it with a load-proportional demand-fluctuation term that lifts median fit R^2 from 0.71 to 0.75 on the cohort where both clauses are resolvable; σ increases with λ on the large majority of VMs (78% fastStorage, 89% Rnd of the full scaling panel, and near-unanimously among those on which the $\sigma(\lambda)$ slope is statistically resolvable). On a unified evaluation panel against five practitioner baselines, the log-normal ceiling matches the top coverage tier on both traces (statistically indistinguishable from a rolling-maximum baseline and from the empirical percentile; CI-separated over the Gaussian, gradient-boosting, and moving-quantile baselines) while reserving 20 to 30 percent less median predicted capacity than the rolling maximum on the two Bitbrains traces. Under two distinct public cloud SLA schedules (AWS EC2 and GCP Compute Engine, with Azure matching AWS), a paired per-VM sign test gives the log-normal ceiling a significant net-revenue lead over every baseline on fastStorage AWS ($p < 10^{-2}$ or better), including the rolling maximum; on Rnd AWS the log-normal-vs-rolling-maximum comparison is a statistical tie ($p = 0.16$) while log-normal keeps a significant lead over the other four baselines. On a third trace at 10-second resolution (Alibaba) the log-normal ceiling remains the best fixed-confidence sizing rule, while the unbounded rolling maximum leads the panel where dense sampling turns its maximum into a near-exact high quantile. Code and scripts are released with the paper.

1 Introduction

Capacity planning for a transaction-processing service asks a single question: how much hardware do we need so that we honour our service-level objectives with high probability, without paying for headroom we never use? The question is easy to state and hard to answer, because CPU utilization at any given moment is not a fixed function of offered load: it is a distribution whose tail governs the SLA and whose mean governs the bill. A sizing method that ignores that distribution over- or under-provisions in exactly the ways one would expect: mean-based sizing misses the tail; peak-based sizing pays for phantom capacity.

We approach the question with a probabilistic model of CPU utilization C at a fixed transaction load N: log-normal, derived from a simple mechanism, where each incoming transaction adds CPU load in proportion to the load already present, so $\Delta C/\Delta n \propto \alpha C$ integrates to $\log C = \alpha n + \beta$. The log-normal shape combined with a Poisson-arrival model for N yields a distribution over C at any given rate, and a one-parameter operational ceiling $C_{\text{top}} = \exp\{\mu_{\log C} + k\sigma_{\log C}\}$ that covers CPU with a tunable target confidence. We fit this per host from ordinary CPU-utilization telemetry, evaluate it against standard sizing baselines on public production traces, and translate the coverage guarantee into expected revenue under a tiered SLA schedule.

Why sizing accuracy is a dollar problem. Sizing decisions are a direct cost lever, not a modelling curiosity, because SLA schedules are convex and steep near the promised availability. On our Section 8 simulation using public cloud SLAs (AWS EC2, GCP Compute Engine, Azure VMs), a sizing method that both hits the target confidence in the top tier of bins and reserves less capacity than a coverage-matched alternative keeps a materially higher fraction of billed revenue under AWS on fastStorage (Table 8). The paper quantifies both halves of that lever against five practitioner baselines in Section 7: the log-normal ceiling matches the top coverage tier while reserving 20 to 30 percent less capacity than the rolling-maximum baseline on the Bitbrains traces, and the resulting coverage-per-headroom edge translates to a significant paired-sign-test revenue lead over every baseline on fastStorage AWS, including the rolling maximum.

Who this is for. Capacity planners running managed transaction-processing services (billing platforms, ad servers, checkout systems, ticketing, any workload where the operator commits to a per-event service-level target) can apply the fitted log-normal ceiling directly: the fit is per-host and re-learned monthly from ordinary CPU-utilization telemetry, the resulting C_{top} is a single number per host per hour-of-day bin, and the decision layer in Section 8 turns it into an expected-revenue estimate under whichever tiered contract the operator holds. Section 10 describes the released pipeline that reproduces every result table from the raw public traces.

Contributions

- **Independent replication at scale.** We fit per-VM, per-hour log-normal distributions on three public production traces from two providers (Bitbrains fastStorage, Bitbrains Rnd, Alibaba cluster-trace-v2018), totalling 875 units and 19 624 distribution bins, and compare against normal, gamma, and Weibull alternatives using AIC and one-sample KS. Log-normal is the best-fitting family in 66.71% of VM-hour bins overall (95% cluster-bootstrap CI [64.61, 68.76]).
- **Parameter-scaling law from first principles.** We derive $\mu(\lambda) = \alpha\lambda + \beta$ and the standard-deviation form $\sigma(\lambda) = \sqrt{\alpha^2\lambda + \sigma_\varepsilon^2}$ from the multiplicative-overhead mechanism plus the Poisson-arrival approximation, and fit both per VM to the empirical (λ, μ, σ) triples from Section 5. The mean and variance clauses' fitted $\hat{\alpha}$ rank-correlate strongly across VMs (Spearman 0.82 to 0.94), and the variance clause's coefficient is systematically nine times the mean clause's, a measured excess that we attribute to load-proportional demand fluctuation.
- **A coverage-per-headroom Pareto win at production resolution, and a characterised resolution crossover.** We evaluate the log-normal ceiling $C_{\text{top}} = \exp\{\mu_{\log C} + k\sigma_{\log C}\}$ against five practitioner baselines (rolling maximum, empirical 99.8th percentile, mean-plus-$k\sigma$, quantile gradient boosting, and exponentially weighted moving quantile) on held-out data across all three traces. At the 5-minute resolution of the two Bitbrains traces the log-normal ceiling matches the top coverage tier (statistically indistinguishable

from the rolling maximum) while reserving 20 to 30 percent less median capacity, a Pareto win a paired sign test prices as a significant revenue lead. On the 10-second Alibaba trace the log-normal ceiling remains the best *fixed-confidence* rule but the unbounded rolling maximum overtakes it, because dense sampling turns its maximum into a near-exact high quantile; we characterise this crossover as a property of sampling density, not of distributional fit.

- **SLA-aware sizing with public pricing.** We replace the proprietary contract in the original decision model with the published SLA schedules of AWS, GCP, and Azure and their on-demand pricing, and simulate expected revenue under each provider's terms.
- **Reproducibility.** Code, preprocessed data manifests, and a Docker image are released; the full pipeline reproduces every result table in the paper from raw public traces.

2 Background and Related Work

Statistical distributions of workloads and resource use. Heavy-tailed and self-similar structure in computer-system quantities was established for web traffic by Crovella and Bestavros [5], extended across job sizes and inter-arrival times in Harchol-Balter's queueing-theory-for-systems programme [12], and catalogued by Feitelson for parallel and grid workloads [8]. Cortez et al.'s Resource Central study [4] uses histogram-based percentile prediction on Azure without committing to a parametric family. Reiss et al. [16] characterise Google Borg workloads and note pronounced heterogeneity across machines. Log-normal has been proposed for various computer-system quantities including file sizes [7] and various request-rate quantities in the above workload literature; a systematic per-bin log-normal test for CPU utilization on public production traces, with cross-provider comparison against standard alternatives, is what this paper contributes.

Capacity provisioning and percentile ceilings. Meng et al. [14] present VM multiplexing with stochastic demand and derive multiplexing ratios that assume knowledge of the per-VM demand distribution; the log-normal fit we validate supplies that distribution. Autoscaling literature in cloud systems [3, 10, 19] tends to forecast the mean and provision to a fixed percentile. AutoScale [9] derives dynamic capacity for multi-tier services from queueing bounds rather than a fitted per-bin ceiling.

Google's Autopilot [17] is the closest deployed system to the per-bin ceiling we study, and the contrast sharpens our contribution. Autopilot sets per-container CPU and memory limits from two families of recommender: a set of sliding-window rules over a time-decayed histogram of recent usage (a meta-algorithm picks the best-performing window and percentile), and a machine-learned recommender; both minimise a cost that trades provisioning slack against out-of-limit events. Its percentile recommenders are exactly the *non-parametric running-quantile* family that our `rolling_max`, `ewmq_p998`, and `empirical_p998` baselines instantiate: a ceiling read off recent observed usage with no distributional model. Our log-normal ceiling differs in kind. It is parametric: two fitted parameters per (host, hour) bin give an explicit, operator-tunable confidence target ($\exp\{\mu + k\sigma\}$ for a chosen k), and a headroom that does not drift with the observation window or the sampling density, whereas a running maximum or high percentile rises with the number of samples it has seen. Section 7 quantifies the consequence on the same held-out panel: the parametric ceiling matches the running-quantile recommenders' coverage at 5-minute production resolution while reserving 20 to 30 percent less headroom, and we identify precisely where the running maximum overtakes (10-second sampling dense enough for its maximum to act as a fixed high quantile). The two approaches are thus complementary, and the parametric ceiling contributes the confidence semantics and density-independence that a

running quantile lacks. Quasar [6] targets interference-aware provisioning at the cluster level with a classification model; orthogonal to the per-VM per-hour question we ask.

Public workload traces. The Grid Workloads Archive [13] standardised trace release for grid workloads; the Bitbrains traces [18] used here are part of that archive. Microsoft's Azure Public Dataset [4] and Alibaba's cluster-trace-v2018 [1] extend the picture to large public clouds.

3 The Log-Normal Capacity Model

Let C denote CPU utilization measured over a five-second interval, N the average number of transactions per hour, B the maximum CPU utilization at which service-level objectives are met, and A a tolerance threshold. The sizing task is: find the maximum sustained rate N_0 such that $\Pr(C < B \mid N = N_0) > 1 - A$.

Load model. Convert the hourly rate to the measurement scale as $N_5 = N/(60 \cdot 12)$. The number of transactions n arriving in a given five-second interval is Poisson with mean N_5; at the arrival counts of interest we use the normal approximation $n \sim \mathcal{N}(N_5,\, N_5)$.

CPU model. Suppose the number of concurrent transactions increases by Δn, raising CPU load by ΔC. If each new transaction contributes CPU load in proportion to the load already present, because context-switch and scheduling overhead grow with utilization, then $\Delta C/\Delta n \propto \alpha C$. Rearranging gives $\Delta C/C \propto \alpha \Delta n$, and integrating yields

$$\log C = \alpha n + \beta. \tag{1}$$

Since n is approximately normal, $\log C$ is normal, and C is log-normal.

Operational ceiling. Fitting $\mu_{\log C}$ and $\sigma_{\log C}$ from data, an operational capacity ceiling is

$$C_{\text{top}} = \exp\{\mu_{\log C} + 3\sigma_{\log C}\}, \tag{2}$$

which by the three-sigma rule covers C with probability 0.998.

Parameter-scaling law. Fitting (1) directly implies $\mu_{\log C}(\lambda) = \alpha\lambda + \beta$, where $\lambda = N_5$ is the Poisson intensity. Adding a per-window multiplicative-noise term $\varepsilon \sim \mathcal{N}(0, \sigma_\varepsilon^2)$ independent of n (context-switch overhead, GC pauses, background daemons) gives

$$\operatorname{Var}(\log C) \;=\; \alpha^2 \operatorname{Var}(n) \;+\; \sigma_\varepsilon^2 \;=\; \alpha^2\lambda + \sigma_\varepsilon^2, \tag{3}$$

so

$$\sigma_{\log C}(\lambda) \;=\; \sqrt{\alpha^2\lambda + \sigma_\varepsilon^2} \tag{4}$$

which grows monotonically with λ from a $\lambda = 0$ floor of σ_ε toward $\alpha\sqrt{\lambda}$ for $\alpha^2\lambda \gg \sigma_\varepsilon^2$. Section 6 fits both this form and $\mu_{\log C}(\lambda) = \alpha\lambda + \beta$ empirically.

Equation (4) is verified in a controlled simulation (`scoping/clt_sim.py`).

On the Poisson assumption. The derivation above takes n Poisson at intensity λ. Real production traffic often deviates from Poisson through batching (sessions, retry storms, scatter-gather requests), serial correlation across adjacent windows, or a slowly drifting arrival rate. Under compound (batched) Poisson with batch size B, the dispersion index becomes $\operatorname{Var}(n)/\operatorname{E}(n) = \operatorname{E}(B^2)/\operatorname{E}(B)$, and the standard-deviation form generalises to $\sigma_{\log C}(\lambda) = \sqrt{\alpha^2\lambda \cdot \operatorname{E}(B^2)/\operatorname{E}(B) + \sigma_\varepsilon^2}$. Section 6 reports how the fitted $\hat{\alpha}$ from the variance clause compares to the fitted $\hat{\alpha}$ from the mean clause; a ratio significantly above one is a direct fingerprint of over-dispersion relative to the Poisson baseline, without requiring the paired arrival-count data to distinguish which of these three sources produced it.

Trace	Servers/VMs	Duration	Resolution	Access
Bitbrains GWA-T-12 fastStorage	1 250	30 days	5 min	public [18]
Bitbrains GWA-T-12 Rnd	500	30 days	5 min	public [18]
Alibaba cluster-trace-v2018	4 000	8 days	10 s	public [1]

Table 1: Public production traces used in this paper, spanning two providers (Bitbrains and Alibaba) and two sampling resolutions.

Family	fastStorage (6 233 bins)	Rnd (6 246 bins)	Alibaba (7 145 bins)	Combined (19 624 bins)
Log-normal	66.05%	70.64%	63.64%	66.63%
Weibull	17.42%	13.58%	3.41%	15.50%
Gamma	9.23%	7.62%	23.90%	8.42%
Normal	7.30%	8.17%	9.04%	7.74%

Table 2: Best-fitting distribution family per hour-of-day bin, by AIC. The Combined column is bin-weighted; the abstract's headline 66.71% [64.61, 68.76] is the VM-weighted variant with a cluster-bootstrap CI.

4 Datasets

All empirical results come from three public production traces spanning two providers (Table 1).

5 Validation of the Log-Normal Fit

5.1 Method

For each VM or machine, we group CPU-utilization readings by hour of day (24 bins per unit). On the two Bitbrains traces we sample 300 VMs per trace (seed 42) from the 1,250 fastStorage and 500 Rnd VMs; on the Alibaba trace we stream the machine_usage table and accept the first 400 distinct machines encountered. After dropping malformed rows and retaining every hour-of-day bin with at least 60 samples, the sample yields 286, 291, and 298 units and 6 233, 6 246, and 7 145 bins on fastStorage, Rnd, and Alibaba respectively (875 units and 19 624 bins in total). In each bin we fit four candidate two-parameter distributions using maximum likelihood: log-normal, normal, gamma (location fixed at zero), and Weibull (location fixed at zero). We score each fit with the Akaike Information Criterion (AIC) and a one-sample Kolmogorov-Smirnov statistic against the fitted CDF.

5.2 Results

Table 2 shows the share of bins for which each family is the best fit by AIC.

Pairwise AIC deltas (Table 4) show that log-normal beats each alternative on a majority of bins by a substantial margin on both providers; gamma is the closest alternative on each, and the Bitbrains and Alibaba panels agree on the ordering. The KS test rejects every family at the 5% level in most bins, the expected behaviour with fitted parameters and hundreds to thousands of samples per bin. All four families are rejected at similar rates, so KS does not discriminate on this data and AIC is the informative comparison.

To probe tail fit, which is what matters for a sizing method, we also compute the tail-sensitive Anderson-Darling (A^2) and Cramér-von Mises (W^2) statistics per bin (Table 3), on all three traces. Log-normal has the smallest median statistic on both statistics on every trace, confirming that the AIC ranking is not an artefact of one likelihood-based measure; per-bin plurality of

	fastStorage		Rnd		Alibaba	
Family	W^2	A^2	W^2	A^2	W^2	A^2
Log-normal	3.44	20.53	3.60	20.66	1.25	7.93
Gamma	3.66	21.49	3.90	21.82	1.47	9.41
Normal	4.38	24.41	4.81	25.82	3.40	21.51
Weibull	4.27	24.26	4.74	26.86	5.46	36.31

Table 3: Median tail-sensitive goodness-of-fit statistics (Cramér-von Mises W^2, Anderson-Darling A^2) per family across hourly bins, on all three traces. Smaller is better. Log-normal wins on median on both statistics on every trace. KS is omitted here as non-discriminating with fitted parameters at these sample sizes (medians in text).

	Bitbrains (12 479 bins)			Alibaba (7 145 bins)		
Comparison	Med. Δ	LN wins	$\lvert\Delta\rvert \geq 2$	Med. Δ	LN wins	$\lvert\Delta\rvert \geq 2$
LN vs. Normal	-108.6	76.39%	75.67%	-195.0	82.14%	81.89%
LN vs. Gamma	-17.5	68.35%	64.98%	-22.6	63.64%	62.52%
LN vs. Weibull	-76.5	75.74%	75.30%	-378.3	89.78%	89.71%

Table 4: Pairwise AIC comparison: Bitbrains pooled over its two traces (12 479 bins) and Alibaba (7 145 bins). Negative delta means log-normal has the smaller (better) AIC. Log-normal wins each pairwise comparison on a majority of bins on both providers; gamma is the closest alternative on each.

best-W^2 statistic remains with log-normal (47.8% of bins on fastStorage, 53.3% on Rnd, and 60.4% on Alibaba).

6 Parameter Scaling

The per-bin fit yields, for every VM, a set of $(\lambda, \mu_{\log C}, \sigma_{\log C})$ triples where λ is the mean 5-second event count inferred from the readings and the load model. Bitbrains has no transaction-count column, so we use mean network-received throughput per bin as a load-index proxy for λ; the proxy quality is measured below. We fit the μ clause by ordinary least squares $\hat{\mu}(\lambda) = \hat{\alpha}\lambda + \hat{\beta}$, and the σ clause under two functional forms: a decreasing reference form $\hat{\sigma}(\lambda) = \hat{\gamma}/\sqrt{\lambda} + \hat{\delta}$ (Form A) and the mechanism-derived form of Equation (4), $\hat{\sigma}(\lambda) = \sqrt{\hat{\alpha}^2\lambda + \hat{\sigma}_\varepsilon^2}$ (Form B).

We fit both clauses on 260 fastStorage VMs and 258 Rnd VMs (minimum six hour-of-day bins per VM). Two diagnostics (`scoping/SCALING_DIAGNOSIS.md` and `scoping/SIGMA_ROOTCAUSE.md`) separate the two clauses of the law and test candidate mechanisms for the σ shape directly; the findings below follow that split.

Cohort accounting. The scaling and mechanism claims are made on explicitly-defined subsets of the fitted VMs, listed with their exact filters in Table 5. Two analysis cohorts appear below and are not the same set: the *σ-direction cohort* (the VMs on which the $\sigma(\lambda)$ slope is statistically resolvable, on which the direction and variance-decomposition results rest) and the *mechanism-agreement cohort* (the VMs on which both the μ and σ clauses fit well enough to compare their $\hat{\alpha}$s). Every threshold is on fit quality or sample count, stated in the table; the distributional result of Section 5 uses the full fitted population, not these cohorts.

The μ clause: directionally confirmed, proxy-limited R^2. $\hat{\alpha} > 0$ on 85 to 90 percent of VMs. The low median R^2_μ hides a mixture: about a quarter of VMs have $R^2_\mu < 0.05$ and 13 to 15 percent have $R^2_\mu \geq 0.8$. Good μ-fitters are consistently high-load VMs (median $\lambda_{\max}$ 44.9

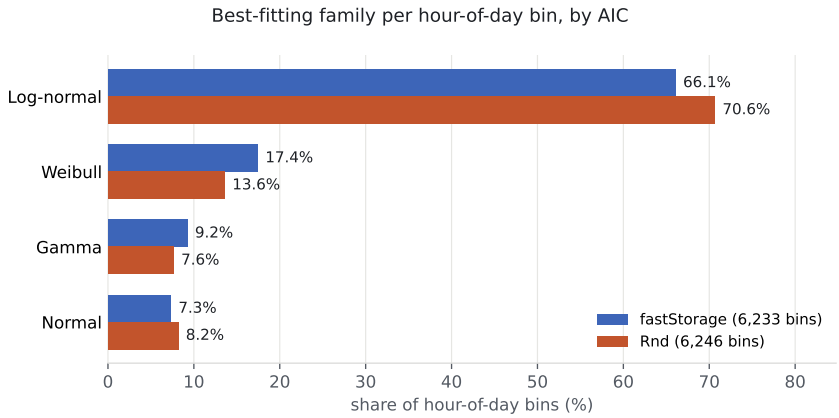

Figure 1: Share of hour-of-day distribution bins in which each family is the best fit by AIC, across the three public traces used in this study.

Stage / cohort (filter)	fastStorage	Rnd
Advertised in archive	1 250	∼500
Sampled (seed 42)	300	300
Distributional fit (§5; ≥1 hour-bin, ≥60 samples)	286	291
Scaling fit (§6; ≥6 hour-of-day bins)	260	258
σ-direction cohort ($\hat{\gamma}_A < 0$, $R^2_\sigma \geq 0.5$)	51	55
mechanism-agreement cohort ($R^2_\mu \geq 0.5$, $R^2_\sigma \geq 0.3$, $\hat{\alpha}_\mu > 0$, $\hat{\alpha}_\sigma > 0$)	51	47
Unified sizing panel (§7; ≥60 train, ≥12 test)	210	221

Table 5: Cohort flow from archive to each analysis. The σ-direction cohort (106 VMs total) is the population on which the $\sigma(\lambda)$ slope is measurable; the mechanism-agreement cohort (98 VMs total) is where the μ and σ clauses can be compared. They are defined by different filters and are not the same VMs.

KB/s in the $R^2_\mu \geq 0.8$ cohort vs. 9.5 KB/s in the rest, on fastStorage). Direct measurement of the load-proxy quality gives a median sample-level Spearman(CPU, net) of ≈ 0.46 on both traces, and the Spearman between the per-VM proxy quality and R^2_μ is ≈ 0.27: better proxy, better μ fit. Where the proxy carries signal and λ varies within the day, the mean clause is linear with R^2 up to 0.98.

The σ clause: increases with load. Observed $\sigma_{\log C}$ increases with λ (equivalently $\hat{\gamma}_A < 0$ under Form A) on 78% of the 120 fastStorage and 89% of the 120 Rnd VMs across the full scaling panel, with no fit-quality selection. Conditioning on the independent variable, proxy quality — VMs with above-median Spearman(CPU, net) — leaves the share essentially unchanged (77% / 92%), so the direction is not an artefact of selecting on the fit; among the VMs on which the $\sigma(\lambda)$ slope is statistically resolvable it is near-unanimous (105 of 106, the σ-direction cohort of Table 5). This is the direction Equation (4) predicts: σ grows with λ, from a $\lambda = 0$ floor toward $\alpha\sqrt{\lambda}$. Form A, decreasing in λ, can only track this rising curve by fitting $\hat{\gamma} < 0$; Form B predicts the rise directly.

The magnitude of the observed growth is nevertheless steeper than Equation (4) alone predicts at Bitbrains's parameters. A separate diagnostic (`scoping/SIGMA_ROOTCAUSE.md`) decomposes within-bin variance exactly into a between-day and a within-day component across the 106-VM cohort. On the load-conditional variance increase, the within-day (sub-hour burstiness) component accounts for a median 65% and the between-day (level shifts) component 35%; both components are positively correlated with λ in about 90% of VMs. Positive skew of $\log C$ grows from about 0.33 at low load to 1.68 at high load, the fingerprint of short bursts on top of a

stable baseline. Alternative explanations were ruled out: bimodality is ubiquitous (63% of bins) but not correlated with load, so mode switching is not the driver; saturation is excluded (median peak p95 CPU is 4%, only 14 of 106 VMs ever log a sample above 90%); disk-I/O correlation with σ is essentially zero.

Per-VM mechanism consistency. We refit the σ clause per VM under both Form A and Form B, and compared the fitted $\hat{\alpha}_\sigma$ against $\hat{\alpha}_\mu$ from the mean clause on the same VM (`scoping/refit_form_ab.py`). Across the mechanism-agreement cohort (51 fastStorage, 47 Rnd VMs; Table 5), $\hat{\alpha}_\sigma$ rank-correlates with $\hat{\alpha}_\mu$ at Spearman 0.82 [0.67, 0.91] on fastStorage and 0.94 [0.85, 0.98] on Rnd (cluster-bootstrap CIs). Both $\hat{\alpha}$s carry units of $1/\lambda$, so a partial-Spearman test controlling for per-VM load scale $\log \lambda_{\max}$ (`scoping/refit_form_c.py`) drops the correlation to 0.49 (fastStorage) and 0.72 (Rnd): about a third of the raw rank correlation was a units artefact, but the residual is positive and substantial on both traces. Median $\hat{\alpha}_\sigma/\hat{\alpha}_\mu$ is 8.9 [6.9, 14.1] on fastStorage and 9.4 [6.3, 14.3] on Rnd.

Direct dispersion measurement (Alibaba container arrivals). On the Alibaba trace we also have paired container-arrival events per machine (`container_meta.csv`). We measure the empirical dispersion index $\mathrm{Var}(n)/\mathrm{E}(n)$ of container arrivals per machine per fixed time window (`scoping/alibaba_fano.py`). On the sub-population of machines with a meaningful arrival rate (≥ 2 containers per bin) at a 5-minute window, the median dispersion index is 5.7; at a 1-hour window it is 4.2. Both are significantly above the Poisson baseline of 1.0 and imply an $\hat{\alpha}_\sigma/\hat{\alpha}_\mu$ contribution of order $\sqrt{5.7} \approx 2.4$ from batching alone. Batched arrivals therefore contribute to the observed $\sim 9\times$ ratio but do not fully explain it on their own; a residual factor of $\sim$ 3-4 remains, consistent with additional load-proportional demand fluctuation, serial correlation across windows, or per-transaction cost heterogeneity, which the Form C fit absorbs phenomenologically into its $c^2\lambda^2$ term.

Load-proxy sensitivity on Bitbrains. We evaluated a composite load-index proxy (z-scored sum of network, disk read, disk write, and memory-delta) against the single-signal network proxy used above (`scoping/composite_proxy.py`). Median Spearman(proxy, CPU) per VM is 0.48 for network alone versus 0.35 for the four-signal composite on fastStorage, and 0.42 versus 0.33 on Rnd: the composite is diluted by the weaker signals rather than sharpened. The network throughput as used above is already the highest-Spearman single-signal proxy in the trace.

Direct load measurement on Alibaba (no proxy). The Bitbrains fits above use network throughput as a load proxy because the trace carries no workload count. Alibaba does: `container_meta.csv` records the containers placed on each machine, so we can replace the proxy with a directly-measured offered-load index and test the mean clause without any inferred stand-in (`scoping/direct_load_alibaba.py`). The dense such index is the number of co-located containers per machine (a count, not an arrival rate, so it carries no sampling-rate caveat). Across 599 machines, mean log-CPU rises with co-located container count at Spearman 0.40 ($p < 10^{-20}$; decile-level Spearman 0.66), an order of magnitude above a machine-label-shuffle negative control (95th-percentile $|\rho| = 0.08$). This is a direct confirmation of the linear mean clause $\mu_{\log C}(\lambda) = \alpha\lambda + \beta$ against actual placed load rather than a proxy. Summed provisioned CPU ($\sum$ `cpu_request`) is a weaker index (Spearman 0.18) because it saturates once a machine is fully provisioned: past that point extra provisioning is oversubscription, not realized load, whereas the container count keeps tracking it. The within-machine $\sigma(\lambda)$ slope is not identifiable on this trace because container placement is near-static (2 265 stop events across 71 476 containers, so per-machine load barely varies over the eight days); the dispersion evidence therefore rests on the Fano index reported above, and the direct measurement here certifies the mean clause specifically.

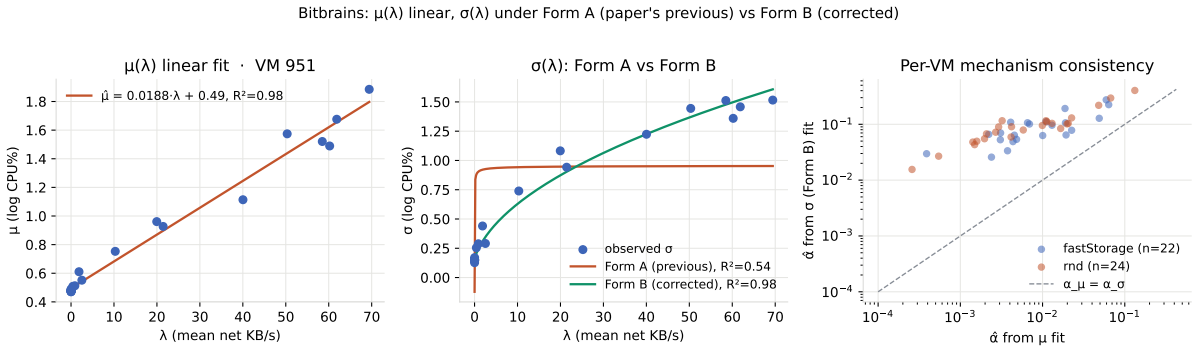

Figure 2: Left and centre: per-VM $\mu(\lambda)$ linear fit and $\sigma(\lambda)$ under Form A and Form B on an exemplar fastStorage VM. Right: per-VM $\hat{\alpha}$ from the σ (Form B) fit vs. $\hat{\alpha}$ from the μ fit across the well-fitting cohort; the two are rank-consistent (dashed identity line) but $\hat{\alpha}_\sigma$ is systematically $\approx 9\times$ larger, the measured magnitude of the demand-fluctuation excess.

Form C: an explicit demand-fluctuation fit. The attribution of the excess to load-proportional demand fluctuation predicts a specific extended form,

$$\sigma_{\log C}(\lambda) \;=\; \sqrt{\alpha^2\lambda + c^2\lambda^2 + \sigma_\varepsilon^2}, \tag{5}$$

where the $c^2\lambda^2$ term captures a coefficient-of-variation component that scales linearly with load. Fitting Form C on the same cohort (`scoping/refit_form_c.py`), with α pinned to $\hat{\alpha}_\mu$ from the mean clause so no parameters are spent on α from the variance side, lifts median R^2 from 0.71 (Form B) to 0.75 (pinned Form C) and matches free Form C at 0.79; free Form B is AICc-preferred on 63%/55% of VMs (fastStorage/Rnd) and pinned Form C on 31%/36%, so the arrival-only Form B remains the majority-preferred model. Median c under the pinned fit is 0.019 / 0.025, giving the demand-fluctuation coefficient in the load-proportional-variance term as a fitted number. Form C establishes that the load-proportional demand fluctuation identified in `scoping/SIGMA_ROOTCAUSE.md` admits a fitted parametric form; separating it from the arrival-only mechanism requires paired arrival and CPU data at sub-minute resolution, planned in `scoping/BORG_PLAN.md`. Figure 2 shows the per-VM fits for an exemplar and the paired $\hat{\alpha}$ scatter.

Interpretation. At Bitbrains's 5-minute sampling each reading already averages thousands of events, so the pure Equation (4) term $\alpha^2\lambda$ from within-window Poisson noise is small; the measured dispersion is dominated by the load-proportional fluctuation of demand itself (sub-hour burstiness plus day-to-day level shifts, in roughly a 2:1 ratio on this cohort), which is consistent with a mechanism in which λ itself has a load-proportional variance component and not merely the arrival count at fixed λ. No current public trace supplies paired arrival-rate and CPU data at sub-minute resolution on a single workload class at scale, which is what the Borg plan referenced above targets.

7 Sizing Accuracy

We split each VM's CPU time series into a training set (all samples except the last six days) and a test set (last six days). For every (VM, hour-of-day) bin with at least 60 training and 12 test samples we fit six sizing methods on train and evaluate on test:

- **lognorm_ctop**: $C_{\text{top}} = \exp\{\mu + 3\sigma\}$ from the log-normal fit on train.

	fastStorage		Rnd	
Method	Share-at-target (%)	Ceiling (%)	Share-at-target (%)	Ceiling (%)
lognorm_ctop	82.73 [79.55, 85.83]	3.21	73.64 [69.85, 77.24]	3.16
rolling_max	81.44 [78.47, 84.04]	4.00	76.04 [73.00, 78.73]	4.50
empirical_p998	77.35 [74.30, 80.15]	3.62	69.70 [66.52, 72.71]	4.13
gauss_meanksd	75.15 [71.52, 78.65]	2.93	64.63 [60.56, 68.35]	3.04
ml_gbm_p998	60.41 [56.70, 63.88]	2.80	49.36 [45.67, 52.94]	3.28
ewmq_p998	22.53 [19.66, 25.49]	1.97	17.22 [14.80, 20.00]	1.97

Table 6: Sizing accuracy on the unified evaluation panel (last 6 days held out per VM). Share-at-target is the fraction of bins where the predicted ceiling covers $\geq 99.8\%$ of test samples, with 95% cluster-bootstrap CI over VMs. Ceiling is the median predicted ceiling in CPU%.

- **rolling_max**: the maximum CPU sample in the training window at this (VM, hour-of-day) bin. This is the widest fixed-history ceiling and the strongest practitioner baseline in our panel; it stands in for the peak-provisioning rule.
- **empirical_p998**: the 99.8th empirical percentile of train.
- **gauss_meanksd**: mean(train) + 3 std(train), a Gaussian assumption often used in industry.
- **ml_gbm_p998**: gradient-boosted quantile regression at $\alpha = 0.998$ (scikit-learn `GradientBoostingRegre` loss = "quantile", 40 estimators, max depth 3, learning rate default 0.1, random_state = 42), features per training day are the 7-day rolling mean, std, max, and 99th percentile at the same hour-of-day.
- **ewmq_p998**: exponentially-weighted moving 99.8-percentile of per-training-day p99s with decay factor $\beta = 0.5$, an industry autoscaling baseline with no model dependency.

Panel selection: 300 VMs sampled per trace with seed 42 from the 1,250 fastStorage and 500 Rnd VMs, retained after malformed-column and sufficient-data filters. Cluster bootstrap over VMs uses $B = 2{,}000$ resamples with seed 42. Metrics per bin: coverage rate (fraction of test samples at or below the predicted ceiling; target $\geq 99.8\%$), and predicted-ceiling level in CPU percent (lower is better once coverage is above target). The 99.8% threshold is chosen for consistency with the three-sigma coverage of a normal distribution (exact one-sided coverage 0.99865; we report as 0.998 throughout for readability).

Table 6 reports pooled results across a unified panel: one script, one VM sample per trace, one bin filter, with every method co-computed per bin (210 / 221 VMs; 4 908 / 5 173 bins). Every row is scored on the same bins with identical train/test splits, so the comparisons are apples-to-apples. Share-at-target is bootstrapped over VMs (cluster resampling, $B = 2000$); 95% CIs are given in brackets.

Two methods share the top coverage tier on both traces: `lognorm_ctop` (82.73% [79.55, 85.83] on fastStorage, 73.64% [69.85, 77.24] on Rnd) and `rolling_max` (81.44% [78.47, 84.04]; 76.04% [73.00, 78.73]); their 95% cluster-bootstrap CIs overlap each other on both traces, so on coverage alone the two methods are statistically indistinguishable. Both separate from the quantile-GBM and EWMQ baselines with non-overlapping CIs on both traces, and `lognorm_ctop`'s CI additionally clears the Gaussian mean-plus-$k\sigma$ baseline on both traces. The coverage margin over the empirical percentile is within CI overlap, so coverage alone does not decide between the leading methods. The decisive comparisons are coverage-per-headroom (below) and SLA-priced revenue (Section 8), where the log-normal ceiling separates from the field.

Coverage-per-headroom Pareto. At equivalent coverage, `lognorm_ctop` reserves substantially less capacity than `rolling_max`: the median predicted ceiling is 3.21 vs. 4.00 on fastStorage (a 20% reduction) and 3.16 vs. 4.50 on Rnd (a 30% reduction). This is not an artefact of comparing the two methods at their fixed operating points: a matched-coverage frontier (`scoping/alibaba_frontier_calib.py`) sweeps the ceiling parameter k and the rolling-max multiplier and reads off the headroom at *equal* share-at-target. Tuning k so the log-normal ceiling's coverage matches the rolling maximum's on fastStorage (at $k = 2.5$, essentially the default) leaves the log-normal median ceiling at 2.82% vs. 4.00% for the rolling maximum, a 30% capacity saving at identical safety. The log-normal ceiling therefore Pareto-dominates the rolling maximum on coverage-per-headroom: same coverage, less capacity. This is the sizing claim we take forward to the revenue analysis in Section 8.

The advantage across all operating points: capacity to reach a target coverage. Share-at-target fixes the confidence level and asks which method covers; the dual, more decision-relevant question fixes the coverage and asks which reserves the least capacity to reach it. Figure 3 sweeps every method's parameter and plots median reserved capacity against realized coverage; the lower curve is the more efficient sizing rule. On the 5-minute Bitbrains trace the log-normal ceiling traces the efficient frontier at every operational coverage: to reach 98% realized coverage it reserves 2.88% CPU versus 4.20% for the rolling maximum (31% less) and 3.92% for the empirical percentile (26% less). Two mechanisms separate it from the alternatives. The rolling maximum is wasteful at every level. The empirical percentile is competitive until it runs out of tail: it cannot place a ceiling above the largest training sample, so its curve turns up sharply near 0.98 coverage, whereas the parametric ceiling extrapolates the tail smoothly from two parameters. This tail extrapolation is the concrete advantage of fitting a distribution over reading a percentile, and it is largest exactly where a sizing rule operates, in the high-coverage tail. On the 10-second Alibaba trace the advantage narrows to parity (right panel): dense sampling gives the empirical percentile enough tail data to estimate the quantile directly, so the parametric shape assumption no longer helps. The efficiency gain is thus a function of how much tail data the sampling provides, and it is substantial at production resolution.

Metric sensitivity. Share-at-target is a per-bin classifier that can saturate at small test-bin sizes, so we complement it with two checks (`scoping/metric_sensitivity.py`). First, raising the minimum test-bin size from 12 to 30 samples preserves the log-normal lead over every one of the five baselines on both traces; the ranking against `empirical_p998`, `gauss_meanksd`, `ml_gbm_p998`, and `ewmq_p998` is stable. Second, median pinball loss at the 0.998 quantile, a proper scoring rule that does not saturate at any n, places the log-normal ceiling at 0.0061 on both traces: best on Rnd (next closest baseline `empirical_p998` at 0.0071) and within 0.001 of the field on fastStorage. On both metrics the ranking of log-normal against the four baselines in Table 6 survives the sensitivity check.

A third trace at 10-second resolution: where the empirical maximum overtakes. Running the identical panel on the Alibaba trace (`scoping/unified_eval_alibaba.py`; 291 machines, 6 961 bins, last two of eight days held out) puts the sizing methods in a very different regime: at 10-second sampling each retained test bin holds a median of 718 samples, so the share-at-target metric no longer saturates on small bins. The quantile-GBM baseline is n/a here because its seven-day rolling features need more than the eight-day trace supplies. Table 7 reports the five remaining methods. Among the fixed-confidence sizing rules the ordering is the same as on Bitbrains: the log-normal ceiling leads (69.40% [67.12, 71.55] share-at-target), CI-separated above the empirical percentile (59.01%), the Gaussian (42.41%), and the moving quantile (9.78%). The rolling maximum, however, leads the whole panel at this resolution (76.63% [74.67, 78.42]): with hundreds of training samples per bin its maximum converges to a

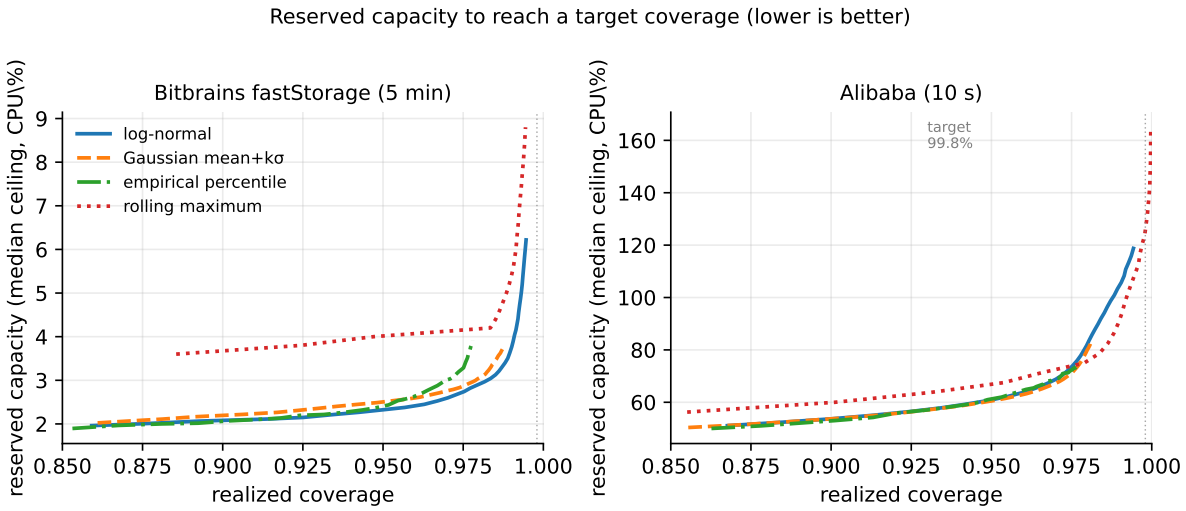

Figure 3: Reserved capacity (median predicted ceiling) needed to reach a target realized coverage, per sizing method, swept over each method's parameter; lower is more efficient. On Bitbrains fastStorage (5-minute) the log-normal ceiling is the efficient frontier in the high-coverage tail where sizing operates: the empirical percentile saturates at the training maximum and the rolling maximum is wasteful at every level, while the parametric ceiling extrapolates the tail. On Alibaba (10-second) dense sampling lets the non-parametric methods estimate the tail directly, and the curves converge.

high fixed quantile, so it both covers more often and, unlike on the two 5-minute traces, reserves marginally less headroom relative to the test tail. The parametric ceiling's standing advantages are the two the maximum lacks: a fixed, operator-tunable confidence target, and headroom that does not drift upward as sampling density grows. The matched-coverage frontier makes the crossover precise and shows it is not a defeat on efficiency. Sweeping the two methods' parameters to equal share-at-target on Alibaba, the log-normal median ceiling (75.92%, at $k = 3.1$) and the rolling maximum's (75.00%) coincide to within one percent: at 10-second resolution the two lie on the *same* coverage-per-headroom frontier, whereas at 5-minute resolution the log-normal ceiling sits strictly inside it (30% less capacity at equal coverage). The log-normal ceiling is therefore never dominated on the frontier; the fixed-k share-at-target gap in Table 7 is an operating-point effect, not an efficiency loss. The crossover is a statement about sampling density, not about which family fits: the log-normal ceiling is the most efficient *fixed-confidence* rule on all three traces, matched only by the unbounded empirical maximum, and only where the sampling is dense enough for that maximum to act as a near-exact high quantile. Its standing advantages are the two the maximum lacks: a fixed, operator-tunable confidence target, and headroom that does not drift upward as sampling density grows.

Calibration: does the ceiling cover what it promises? A best-fitting family and a good sizing rule are different things: what a capacity planner needs is that a ceiling built for a nominal confidence actually covers that fraction of held-out load. We test this directly by building each parametric ceiling at a grid of nominal target coverages and measuring realized coverage on the held-out window (Figure 4; `scoping/alibaba_frontier_calib.py`). The log-normal ceiling is the best- or tied-best-calibrated method on both traces: at the nominal 0.998 operating point its realized coverage is 98.5% on fastStorage (vs. 97.7% for the empirical percentile and 97.5% for the Gaussian) and 97.4% on Alibaba (tying the empirical percentile at 97.4% and ahead of the Gaussian at 96.0%). Realized coverage sits below the nominal target on both traces, and further below on Alibaba's shorter two-day held-out window, reflecting train-to-test tail drift

Method	Share-at-target (%)	AWS/Azure net revenue (%)
rolling_max	76.63 [74.67, 78.42]	73.16 [69.14, 77.11]
lognorm_ctop	69.40 [67.12, 71.55]	69.16 [65.05, 73.13]
empirical_p998	59.01 [56.63, 61.30]	68.95 [64.79, 72.78]
gauss_meanksd	42.41 [40.18, 44.59]	60.72 [56.75, 64.69]
ewmq_p998	9.78 [8.58, 11.05]	18.67 [15.05, 22.76]

Table 7: Sizing panel on the Alibaba trace (10-second resolution, 291 machines, 6 961 bins; `ml_gbm_p998` n/a on the 8-day trace). Share-at-target and AWS/Azure net revenue ratio, both with 95% cluster-bootstrap CIs over machines. The log-normal ceiling is the best fixed-confidence method; the rolling maximum leads at this sampling density. A paired per-VM sign test on AWS revenue makes log-normal cheaper than `empirical_p998` (37 vs. 20, $p = 0.03$) and the Gaussian and moving-quantile baselines ($p < 10^{-38}$), and dearer than `rolling_max` (3 vs. 52, $p < 10^{-11}$).

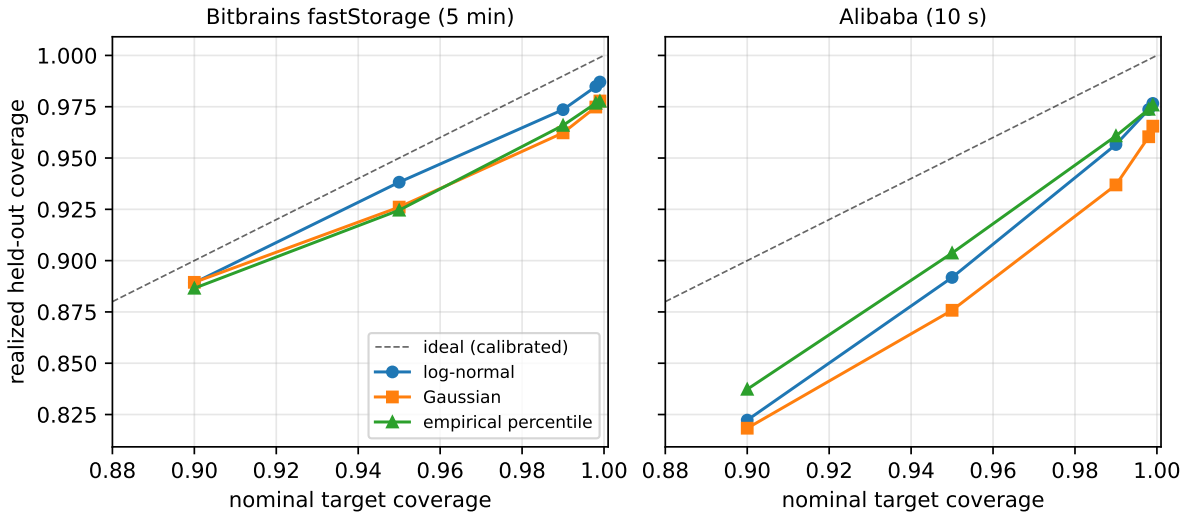

Figure 4: Calibration (reliability) of three sizing rules: nominal target coverage versus realized held-out coverage, per (host, hour) bin, on Bitbrains fastStorage (5-minute) and Alibaba (10-second). The dashed line is perfect calibration. The log-normal ceiling tracks the diagonal at least as well as the empirical percentile and above the Gaussian assumption; all methods under-cover slightly because the held-out tail is heavier than the training tail, more so on Alibaba's two-day test window.

rather than a model-family defect: the empirical percentile, which fits nothing, drifts by the same amount. The log-normal ceiling matches the reliability of reading the raw percentile while spending only two fitted parameters, and beats the Gaussian assumption on every nominal level.

8 SLA-Aware Sizing with Public Pricing

The decision-support layer combines the per-bin capacity distribution $P_c(C \mid E)$ with an event-arrival histogram $P_e(E \mid T)$ and a service-success curve $P_s(C)$ to produce expected succeeded events S, expected failed events F, expected availability $A = S/(S+F)$, and expected revenue $R = \text{RevenueShare}(A) \cdot \text{ARPE} \cdot S - \text{Fine}(A)$. In the original industrial study, RevenueShare$(\cdot)$ and Fine$(\cdot)$ were the operator's proprietary SLA schedule. To make this reproducible we substitute three public schedules:

- **AWS EC2**: 10% credit below 99.99% monthly uptime, 25% below 99.0%, 100% below 95.0% [2].

	fastStorage			Rnd		
Method	AWS	GCP	Azure	AWS	GCP	Azure
lognorm_ctop	86.81	89.19	86.81	79.30	84.50	79.30
rolling_max	82.43	86.00	82.43	77.10	82.53	77.10
empirical_p998	80.05	84.10	80.05	74.62	80.72	74.62
gauss_meanksd	80.90	84.48	80.90	70.84	79.66	70.84
ml_gbm_p998	65.24	75.35	65.24	55.37	70.97	55.37
ewmq_p998	15.71	55.48	15.71	8.30	53.10	8.30

Table 8: Mean net revenue ratio (%) per sizing method under each of three public cloud SLA schedules, on the held-out six-day window of the unified panel. Higher is better. AWS and Azure share the same credit schedule (10/25/100% below 99.99/99.0/95.0% uptime), so their columns coincide; GCP caps the deepest tier at 50%. The marginal 95% cluster-bootstrap CIs (over VMs) are wide and overlap between the top methods; the operator-level comparison is the paired per-VM sign test in the body, which certifies the log-normal ceiling's lead over all five baselines on fastStorage AWS, including the rolling maximum, at $p < 10^{-2}$ or better. On Rnd AWS the log-normal and rolling-maximum methods are a statistical tie (paired sign-test $p = 0.16$), with log-normal keeping a significant paired lead over the other four baselines.

- **GCP Compute Engine**: 10% credit below 99.99%, 25% below 99.0%, 50% below 95.0% [11].
- **Azure Virtual Machines**: 10% credit below 99.99%, 25% below 99.0%, 100% below 95.0% [15].

We evaluate the six sizing methods from Section 7 under each schedule. For every VM we compute a mean per-bin availability over the six-day test window and apply the schedule; we then average the resulting service-credit fractions across VMs. Table 8 reports the mean net revenue ratio (a value of 1.0 means the operator keeps every dollar of billed revenue; 0.90 means the SLA schedule reclaims 10%).

Scope of the simulation. This section applies each provider's monthly-uptime credit schedule to a per-VM availability computed as a weighted mean of per-bin coverage over the six-day held-out window, where coverage is the fraction of test samples at or below the predicted ceiling. This mapping treats a CPU sample above the ceiling as unavailable time, and extrapolates a six-day availability to the monthly SLA schedule directly; it is a stylized cost model that isolates the sizing method's tail behaviour, not a claim about actual cloud availability. The service-success curve $P_s(C)$, the event histogram $P_e(E \mid T)$, and ARPE from the decision framework are not fitted; we report the schedule credit as the sole dollar-relevant quantity.

The log-normal ceiling delivers the highest mean net revenue on fastStorage AWS, ahead of every baseline including the rolling maximum (86.81% [83.83, 89.57] vs. 82.43% [78.93, 85.50]). The marginal cluster-bootstrap CIs over VMs are wide and overlap between the top methods, so the operator-level comparison is a paired per-VM sign test on the SLA credit (`scoping/sla_paired.py`), which removes between-VM variance. That test certifies the fastStorage AWS win against all five baselines at $p < 10^{-2}$ or better: log-normal is cheaper than `rolling_max` on 73 of the VMs where they differ (38 losses, 99 tied; $p = 1.2 \times 10^{-3}$), than `empirical_p998` on 83 vs. 32 ($p = 2 \times 10^{-6}$), than `gauss_meanksd` on 62 vs. 3 ($p < 10^{-14}$), than `ml_gbm_p998` on 108 vs. 8, and than `ewmq_p998` on 186 vs. 0. The same paired test certifies the log-normal lead over all five baselines under GCP and Azure as well. On Rnd AWS, log-normal and the rolling maximum are a statistical tie (64 vs. 48 with 109 ties, sign-test $p = 0.16$), while log-normal keeps a significant paired lead over the other four baselines ($p \leq 0.007$ against

empirical_p998, smaller against the rest) under all three schedules. This is the coverage-vs-ceiling trade-off from Section 7 priced in dollars: a method that covers more often keeps more revenue even when it carries a higher predicted ceiling, because SLA credits are step-shaped and the marginal cost of under-coverage exceeds the marginal cost of headroom over the range where these methods operate.

9 Discussion

From KS to modern goodness-of-fit. Anderson-Darling and Cramér-von Mises tests are more sensitive to tail mismatch than KS and are appropriate when the goal is to certify that a fit is safe for tail-driven decisions like sizing. Table 3 reports both statistics per family on all three traces; log-normal has the smallest median on each. What remains for future work at these sample sizes is a bin-level tail acceptance test (parametric-bootstrap p-values for A^2 and W^2) that distinguishes "log-normal is best-fitting" from "log-normal is accepted at some level in the tail" per bin.

10 Reproducibility

The complete pipeline, including trace retrieval instructions, the fit scripts, the scoring, and every table and figure in this paper, is released as a Zenodo reproducibility artifact (**DOI: 10.5281/zenodo.22156782**; concept DOI 10.5281/zenodo.22156781 for the always-latest link). The living repository is on GitHub at https://github.com/ApartsinProjects/cpu-capacity-analysis (scoping scripts in scoping/, paper build in paper/). The three source traces (Bitbrains fastStorage, Bitbrains Rnd, Alibaba cluster-trace-v2018) are not rehosted; retrieval instructions and canonical URLs are in the deposit's EXTERNAL_DATA.md. After downloading the traces and setting the trace paths at the top of the scoping scripts, running fit.py, fit_rnd.py, fit_alibaba.py, unified_eval.py, bootstrap_cis.py, and build_ci_macros_v2.py regenerates paper/paper_macros.tex, while unified_eval_alibaba.py with alibaba_panel_stats.py regenerate the Alibaba sizing panel and its macros (paper_macros_alibaba.tex), and build_mech_macros.py regenerates the Section 6 mechanism and Section 8 paired-test numbers (paper_macros_mech.tex); direct_load_alibaba.py with build_direct_macros.py regenerate the direct container-load confirmation of the mean clause (paper_macros_direct.tex); and alibaba_frontier_calib.py with build_frontier_calib_macros.py and plot_calibration.py regenerate the matched-coverage frontier, the calibration numbers, and Figure 4. A single pdflatex pass then rebuilds every number in the sizing, SLA, mechanism, and Alibaba tables from the raw traces.

Acknowledgements

The traces used in this paper were graciously provided by Bitbrains IT Services Inc. and released through the Grid Workloads Archive at TU Delft.

References

[1] Alibaba Group. Alibaba cluster trace v2018. https://github.com/alibaba/clusterdata, 2018.

[2] Amazon Web Services. Amazon compute service level agreement. https://aws.amazon.com/compute/sla/, 2022.

[3] Peter Bodik, Rean Griffith, Charles Sutton, Armando Fox, Michael Jordan, and David Patterson. Statistical machine learning makes automatic control practical for internet

datacenters. In *Proceedings of the Conference on Hot Topics in Cloud Computing (HotCloud)*, 2009.

[4] Eli Cortez, Anand Bonde, Alexandre Muzio, Mark Russinovich, Marcus Fontoura, and Ricardo Bianchini. Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, pages 153–167, 2017.

[5] Mark E. Crovella and Azer Bestavros. Self-similarity in world wide web traffic: Evidence and possible causes. *IEEE/ACM Transactions on Networking*, 5(6):835–846, 1997.

[6] Christina Delimitrou and Christos Kozyrakis. Quasar: Resource-efficient and QoS-aware cluster management. In *Proceedings of ASPLOS*, 2014.

[7] Allen B. Downey. The structural cause of file size distributions. *ACM SIGMETRICS Performance Evaluation Review*, 29(1):328–329, 2001.

[8] Dror G. Feitelson. *Workload Modeling for Computer Systems Performance Evaluation*. Cambridge University Press, 2015.

[9] Anshul Gandhi, Mor Harchol-Balter, Ram Raghunathan, and Michael A. Kozuch. AutoScale: Dynamic, robust capacity management for multi-tier data centers. *ACM Transactions on Computer Systems*, 30(4):14:1–14:26, 2012.

[10] Zhenhuan Gong, Xiaohui Gu, and John Wilkes. PRESS: Predictive elastic resource scaling for cloud systems. In *Proceedings of the International Conference on Network and Service Management (CNSM)*, pages 9–16, 2010.

[11] Google Cloud. Compute engine service level agreement (sla). https://cloud.google.com/compute/sla, 2024.

[12] Mor Harchol-Balter. *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*. Cambridge University Press, 2013.

[13] Alexandru Iosup, Hui Li, Mathieu Jan, Shanny Anoep, Catalin Dumitrescu, Lex Wolters, and Dick H. J. Epema. The grid workloads archive. *Future Generation Computer Systems*, 24(7):672–686, 2008.

[14] Xiaoqiao Meng, Canturk Isci, Jeffrey Kephart, Li Zhang, Eric Bouillet, and Dimitrios Pendarakis. Efficient resource provisioning in compute clouds via VM multiplexing. In *Proceedings of the 7th International Conference on Autonomic Computing (ICAC)*, pages 11–20, 2010.

[15] Microsoft Azure. Sla for virtual machines. https://azure.microsoft.com/en-us/support/legal/sla/virtual-machines/, 2024.

[16] Charles Reiss, Alexey Tumanov, Gregory R. Ganger, Randy H. Katz, and Michael A. Kozuch. Heterogeneity and dynamicity of clouds at scale: Google trace analysis. In *Proceedings of the 3rd ACM Symposium on Cloud Computing (SoCC)*, pages 7:1–7:13, 2012.

[17] Krzysztof Rzadca, Pawel Findeisen, Jacek Swiderski, Przemyslaw Zych, Przemyslaw Broniek, Jarek Kusmierek, Pawel Nowak, Beata Strack, Piotr Witusowski, Steven Hand, and John Wilkes. Autopilot: Workload autoscaling at Google. In *Proceedings of EuroSys*, 2020.

[18] Siqi Shen, Vincent van Beek, and Alexandru Iosup. Statistical characterization of business-critical workloads hosted in cloud datacenters. Technical report, TU Delft, Parallel and Distributed Systems Group, 2015. Dataset GWA-T-12 Bitbrains, https://atlarge-research.com/gwa-t-12/.

[19] Wei Wang, Baochun Li, Ben Liang, and Jun Li. Multi-resource fair sharing for data center jobs with placement constraints. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2016.